

HW 8

1

For the circuit shown in Figure P1, compute the RC delay from node Out to $Z1$, $Z2$, and $Z3$ using the Rubenstein-Penfield model. Assume that $R1 = R2 = 5k\Omega$, $R3 = R4 = 10k\Omega$, and $R5 = R6 = R7 = 15k\Omega$, and all capacitors are equal to $0.02pF$.

✓ Answer ✓

$$T_{Z1} = 8R_1C + R_3C + (R_3 + R_4)C = 1.4 \text{ ns}$$

$$T_{Z2} = 8R_1C + R_2C = 0.9 \text{ ns}$$

$$T_{Z3} = 8R_1C + R_7C + (R_7 + R_5)C + (R_7 + R_5 + R_6)C = 2.6 \text{ ns}$$

2

The circuit in Figure P2 implements a 1-bit full adder. Assume that all transistors have an ON-resistance $R = 10,000 \Omega$, a drain capacitance of $C_d = 0.001 \text{ pF}$, and a gate capacitance $C_g = 0.01 \text{ pF}$.

a

Compute the delay from input to output for both the sum and the carry outputs when $A = B = C = 1$ using (1) the Lump RC model and (2) Rubenstein-Penfield model. Compare your results.

✓ Answer

After converting the diagram into an RC model, our pathes to the outputs reduce to `A→inv->cap→2 parallel R→cap`

$$T = RC = 10000(0.001) \text{ ps} = 10 \text{ ps}$$

With the lumped model we would get

$$T = 2RC = 20 \text{ ps}$$

b

The following input sequence $ABC = (100), (011), (111), (000)$ is applied at the primary input of the adder (Assume initial values at $(ABC) = (111)$). Compute the number of changes on output nodes S and C_{out} .

✓ Answer

Input	Output
111	11

Input	Output
100	10
011	01
111	11
000	00

Both change state 3 times.

c

Repeat (b) for this sequence: $ABC = (001), (111), (011), (100)$.

✓ Answer

Input	Output
111	11
001	10
111	11
011	01
100	10

S changes twice and C_{out} changes 3 times.

d

Compare and comment on the result of part (b) and (c)

✓ Answer

The order of the switches dictates the number of switches total out the output.

3

A CMOS inverter is designed with $\beta_n = 2.5\beta_p$, $V_{tn} = +1$ V, and $V_{tp} = -0.9$ V. Compute V_{IL} , V_{IH} , and V_{th} if $V_{dd} = 5$ V.

✓ Answer

NMOS: $V_{GSn} = V_{in}$, $V_{DSn} = V_{out}$

PMOS: $V_{GSp} = V_{in} - V_{dd}$, $V_{DSp} = V_{out} - V_{dd}$

Both NMOS and PMOS are saturated at the switching threshold.

$$\beta_n(V_{th} - V_{tn})^2 = \beta_p(V_{th} - V_{dd} - V_{tp})^2$$

$$\sqrt{2.5\beta_p}(V_{th} - V_{tn}) = \sqrt{\beta_p}(V_{dd} + V_{tp} - V_{th})$$

$$\begin{aligned}
V_{th}\sqrt{2.5\beta_p} - V_{tn}\sqrt{2.5\beta_p} &= V_{dd}\sqrt{\beta_p} + V_{tp}\sqrt{\beta_p} - V_{th}\sqrt{\beta_p} \\
V_{th}\sqrt{2.5\beta_p} + V_{th}\sqrt{\beta_p} &= V_{dd}\sqrt{\beta_p} + V_{tp}\sqrt{\beta_p} + V_{tn}\sqrt{2.5\beta_p} \\
V_{th} &= \frac{\sqrt{\beta_p}(V_{dd}+V_{tp})+V_{tn}\sqrt{2.5\beta_p}}{\sqrt{2.5\beta_p}+\sqrt{\beta_p}} \\
V_{th} &= \frac{V_{dd}+V_{tp}+\sqrt{2.5}V_{tn}}{\sqrt{2.5}+1} \\
&= 2.20102024884066 \text{ V}
\end{aligned}$$

For V_{IL} and V_{IH} I utilized a linear solver to not need to do the algebra myself.

```

from sympy import *
from sympy.matrices import Matrix

vin, vout, vdd = symbols("Vin Vout Vdd")
bp, bn = symbols("\\beta_p \\beta_n")
vg sn, vdsn, vtn = symbols("Vgsn Vdsn Vtn")
vg sp, vdsp, vtp = symbols("Vgsp Vdsp Vtp")
vth, vil, vih = symbols("Vth VIL VIH")

mos = {
    vgsn: vin,
    vdsn: vout,
    vg sp: vin - vdd,
    vdsp: vout - vdd
}

given = {
    bn: 5/2*bp,
    vtn: 1,
    vtp: -0.9,
    vdd: 5
}

linear = lambda b, vgs, vt, vds: b * ((vgs - vt)*vds - vds**2 / 2)
saturated = lambda b, vgs, vt: b/2*(vgs-vt)**2

f = saturated(bn, vgsn, vtn) - saturated(bp, vg sp, vtp)
vth = solve(f.subs(mos).subs(given), vin)

f = saturated(bn, vgsn, vtn) - linear(bp, vg sp, vtp, vdsp)
vos = solve(f.subs(mos).subs(given), vout)
vil = [solve(diff(v, vin) + 1, vin) for v in vos]

f = linear(bn, vgsn, vtn, vdsn) - saturated(bp, vg sp, vtp)
vos = solve(f.subs(mos).subs(given), vout)
vih = [solve(diff(v, vin) + 1, vin) for v in vos]

```



$$V_{IL} = 1.72002609818140 \text{ V}$$

$$V_{IH} = 2.47763742627852 \text{ V}$$

4

Find V_{IL} , V_{IH} and V_{th} for a CMOS inverter with $V_{tn} = 0.8 \text{ V}$, $V_{tp} = -0.8 \text{ V}$, $(W/L)_n = 1.4$ and $(W/L)_p = 4$. The power supply voltage is $V_{dd} = 5 \text{ V}$

✓ Answer

Like question 3, I utilized a linear solver to obtain our values:

```
from sympy import *
from sympy.matrices import Matrix

vin, vout, vdd = symbols("Vin Vout Vdd")
vgsn, vdsn, vtn = symbols("Vgsn Vdsn Vtn")
vgsp, vdsp, vtp = symbols("Vgsp Vdsp Vtp")
vth, vil, vih = symbols("Vth VIL VIH")
mun, mup, ln, lp, wn, wp, tox = symbols("mun mup Ln Lp Wn Wp tox")

mos = {
    vgsn: vin,
    vdsn: vout,
    vgsp: vin - vdd,
    vdsp: vout - vdd
}

given = {
    mun: 2 * mup,
    bn: 5/2 * bp,
    vtn: 0.8,
    vtp: -0.8,
    wn: 1.4 * ln,
    wp: 4 * lp,
    vdd: 5
}

b = lambda mu, w, l: mu*w/tox/l
linear = lambda b, vgs, vt, vds: b * ((vgs - vt)*vds - vds**2 / 2)
saturated = lambda b, vgs, vt: b/2*(vgs-vt)**2

bn = b(mun, wn, ln)
bp = b(mup, wp, lp)

f = saturated(bn, vgsn, vtn) - saturated(bp, vgsp, vtp)
```

```

vth = solve(f.subs(mos).subs(given), vin)

f = saturated(bn, vgsn, vtn) - linear(bp, vgsp, vtp, vdsp)
vos = solve(f.subs(mos).subs(given), vout)
vil = [solve(diff(v, vin) + 1, vin) for v in vos]

f = linear(bn, vgsn, vtn, vdsn) - saturated(bp, vgsp, vtp)
vos = solve(f.subs(mos).subs(given), vout)
vih = [solve(diff(v, vin) + 1, vin) for v in vos]

```



$$V_{th} = 2.65118636594714 \text{ V}$$

$$V_{IL} = 2.27426637944763 \text{ V}$$

$$V_{IH} = 3.12167222994657 \text{ V}$$